

PujaConnect

Online Pandit & Booking Platform

PROJECT REPORT

Technical Documentation & Architecture Manual

Submitted by:

ABHIRAMA H Y

abhiramhy836@gmail.com

PES University

Under the Guidance of:

Unified Mentor

Program Manager

ABSTRACT

In the modern era, while digital solutions have disrupted retail, travel, and logistics, the religious services sector in India remains highly unorganized and fragmented. Booking a Pandit (priest) for sacred rituals currently relies heavily on word-of-mouth recommendations, local references, or direct temple visits. This lack of standardization leads to opaque pricing, scheduling conflicts, and difficulty in finding credentialed, verified professionals.

PujaConnect is a premium, end-to-end full-stack digital platform designed to bridge this gap. Built using the MERN stack (MongoDB, Express, React, Node.js) and styled with a customized, Karnataka-inspired saffron and gold palette, PujaConnect establishes a reliable ecosystem connecting devotees (Users), priests (Pandits), and administrators (Admins).

Devotees benefit from transparent slot-based booking, an intuitive interface, direct booking-related chat, and detailed profiles. Pandits are empowered with digital tools to set custom availability, list services with standard pricing, and manage requests. Admins oversee the platform quality through manual credentials verification, ritual catalog curations (100+ categories), and user account moderations.

This report details the design decisions, system architecture, database design, core scheduling mechanics (including buffer gaps and slot locking), API parameters, security guidelines, and deployment practices (Vercel, Render, MongoDB Atlas, Cloudinary) that make PujaConnect a production-ready, highly scalable, and secure application.

Live Deployed Application: <https://puja-connect-ahy.vercel.app/>

TABLE OF CONTENTS

1. Project Overview	Page 4
2. Introduction	Page 5
3. Problem Statement	Page 6
4. Existing System	Page 9
5. Proposed Solution	Page 10
6. Objectives	Page 11
7. Scope	Page 13
8. Complete System Architecture	Page 14
9. Technology Stack	Page 15
10. Folder Structure	Page 17
11. Database Design	Page 19
12. Authentication Flow	Page 26
13. User (Devotee) Module	Page 28
14. Pandit (Priest) Module	Page 30
15. Admin Module & Moderation	Page 32
16. Ritual Management	Page 34
17. Booking Workflow	Page 35
18. Availability Management	Page 36
19. Messaging System	Page 37
20. Search & Filtering	Page 38
21. Admin Verification Workflow	Page 39
22. Deployment Architecture	Page 40
23. API Overview	Page 41
24. Security Features	Page 43
25. Future Enhancements	Page 44
26. Challenges Faced	Page 45
27. Conclusion	Page 47
28. Project Screenshots Guide	Page 48

1. Project Overview

Live Deployed Application Link:

<https://puja-connect-ahy.vercel.app/>

PujaConnect is a production-ready, full-stack digital booking platform built using the MERN stack (MongoDB, Express, React, Node.js). Styled with a customized, Karnataka-inspired saffron and gold palette, it creates an organized, transparent, and trusted digital ecosystem bridging devotees (Users), religious service providers (Pandits), and platform managers (Admins).

The platform includes structural components for slot-based real-time calendar availability, direct booking-linked message threads with unread counters, in-memory Cloudinary image upload streams, manual credentials verification workflows, and strict role-based route protections.

The system is designed to be horizontally scalable. Each core concern (authentication, booking transactions, availability calculations, and messaging) is implemented in its own controller file, allowing independent modification without side effects across other modules.

Ecosystem Core Parameters

- Ecosystem Roles: User (Devotee), Pandit (Priest), and Admin
- Access Control: Three distinct dashboard layouts with Role-Based Access Control (RBAC)
- Catalog Size: 100+ Sacred Hindu Rituals across 12 custom categories
- Key Tech Stack: MERN (MongoDB Atlas, Express.js, React.js, Node.js) + Tailwind CSS

2. Introduction

In Hindu traditions, planning religious ceremonies is a sacred task, yet finding the right Pandit who is certified, speaks the appropriate regional language, and understands specific custom variations remains highly unorganized. PujaConnect resolves this digital divide by offering a unified discovery portal where devotees can discover Pandits, examine pricing models, and coordinate materials.

This project represents a complete full-stack answer to the digitization of religious bookings, establishing standards in time-scheduling availability, secure communication channels, and identity validations.

Platform Core Pillars

- **Multi-criteria Discovery:** Users search verified Pandit profiles by city, state, experience, and spoken languages, eliminating the trust barrier.
- **Pricing Transparency:** Explicit pricing models, required material checklists, estimated durations, and preparation guidelines are displayed upfront.
- **Unified Coordination:** Dedicated messaging threads are created dynamically for each booking, allowing devotees and priests to coordinate gotra, addresses, and materials.

3. Problem Statement

3.1 Industry Context

In India, the religious services sector, specifically for Vedic rituals, pujas, and homas, is a massive market. However, it operates almost entirely within an informal, unorganized, and highly fragmented ecosystem. Traditionally, devotees rely on family networks, word-of-mouth recommendations, or local temple priests to find a Pandit (priest) for sacred occasions. This informal network lacks standards in credentials, pricing, and scheduling.

3.2 Devotee Pain Points

- **Trust and Verification Deficit:** There is no centralized registry to verify a priest's certifications (e.g., Veda Pathshala degrees) or background, leaving devotees uncertain about the Pandit's authenticity.
- **Opaque and Inconsistent Fees:** The lack of standardized pricing leads to arbitrary negotiations. Devotees face opaque costs that vary wildly based on the location, priest, or auspiciousness of the day.
- **Coordination and Preparation Gaps:** Coordinating gotra details, required puja preparation checklists (Samagri), and exact timing depends on fragile phone calls, leading to misunderstandings and preparation delays.
- **Scheduling Friction:** Devotees frequently face last-minute cancellations, scheduling conflicts, or unavailability of priests during major festival seasons (Muhurats).

3.3 Pandit Pain Points

- **Digital Exclusion:** Many qualified priests are digitally offline, making them invisible to a wider metropolitan audience beyond their immediate community circles.
- **Manual Calendar Management:** Relying on handwritten diaries leads to scheduling overlaps, double-bookings, and lost economic opportunities.
- **Unequal Market Access:** Priests who lack prominent temple affiliations or established family networks face significant barriers in discovering client requests, resulting in uneven work opportunities.

3.4 Product Scope & Technical Goals

PujaConnect implements a robust marketplace where authenticated devotees discover, search, and book admin-verified Pandits. The platform scope and features address the core industry constraints:

- In Scope (Phase 1): Responsive web client (desktop/mobile), verified Pandit profiles, structured ritual catalog, slot-based booking engine, direct chat coordinator, and admin moderation dashboards.
- Out of Scope (Phase 2): Dedicated native mobile applications, automated Muhurat calculations, live-streamed online pujas, and global multi-language translation support.

3.5 Core Data Entities

To resolve the problem technically, the system implements a relational-first document design centered on five core entities:

- Users: Credentials, contact details, and role categories (user, pandit, admin).
- Pandits: Experience records, location parameters, pricing maps, and verification states.
- Rituals: Standardized catalog details, occasion descriptions, durations, and checklists.
- Bookings: Chronological parameters, location types, status tracking, and message lists.
- Availability: Daily calendar sheets mapping active timeslots and booking associations.

3.6 Non-Functional Requirements (NFRs)

- Performance: Fast page load times (<3 seconds) across search grids and dashboards.
- Security: Stateless JWT session verification, passwords encrypted with bcrypt (12 rounds), and route-level Role-Based Access Control (RBAC).
- Usability: Clean, culturally respectful, and clear-to-navigate interface optimized for multiple age groups.
- Scalability: Scalable schema structure designed to support multi-city and multi-region listings.
- Reliability: Database transaction logic to prevent duplicate bookings or slot overlapping.

3.7 Key Performance Indicators (KPIs)

- Devotees Onboarded: Total registered user accounts.
- Active Priests: Number of verified and listed Pandit profiles.
- Fulfillment Rate: Ratio of completed bookings to requested bookings.
- Scheduling Efficiency: Reduction in manual coordination time through slot locking.

4. Existing System

The traditional religious booking workflow relies on unverified contacts, local temple visits, or word-of-mouth networks. It has no central reference registry, pricing transparency, or booking guarantees.

Key Failures of the Existing System

- **Lack of Digital Presence:** Pandits lack online profiles or calendars, making them invisible to potential clients beyond their immediate local community.
- **Double-Booking Risk:** Priests maintain handwritten diaries, leading to frequent scheduling collisions, double-bookings, and last-minute cancellations.
- **Opaque Pricing:** No standardized rate cards exist. Fees vary wildly and are negotiated ad-hoc, creating trust issues and confusion.
- **No Verification:** Credentials, certifications, and experience claims are impossible to verify through informal channels.

5. Proposed Solution

PujaConnect implements a robust marketplace where authenticated devotees discover, search, and book admin-verified Pandits. Pricing maps are explicitly set per ritual. Availability schedules are managed via active date lists containing distinct slot objects. Coordination is handled through booking-linked message lines, ensuring transparent communication.

Core Solution Capabilities

- **Verified Listings:** Every Pandit undergoes manual admin verification before appearing in public search results, creating a trustworthy network.
- **Slot-Based Calendar:** Pandits define availability as time slots; conflict-checking middleware prevents double-bookings automatically.
- **Fixed Pricing Maps:** Each Pandit sets explicit per-ritual pricing stored as a Mongoose Map, eliminating ad-hoc negotiations.
- **Booking Chat:** Embedded messaging arrays inside each booking document enable direct, contextual coordination.

6. Objectives

The primary objective of PujaConnect is to bring organization, transparency, and digital efficiency to the traditional religious booking ecosystem in India. By utilizing modern web technologies, the platform aims to empower both devotees and priests through verified interactions, seamless scheduling, and structured pricing models.

Historically, religious bookings in Karnataka and wider India have operated through informal, word-of-mouth networks. This lack of structure leads to scheduling conflicts, pricing discrepancies, and difficulty for devotees in verifying a priest's credentials. PujaConnect addresses these challenges directly by providing a formal marketplace that preserves Vedic traditions while introducing modern reliability.

Digitization & Transparency Goals

- **Verified Professional Network:** Establishing a trustworthy directory of religious service providers by manually verifying their certifications (Veda Pathshala, academic degrees in Sanskrit), years of experience, and location credentials.
- **Pricing Transparency:** Eliminating arbitrary pricing fluctuations and ad-hoc negotiations. The platform records explicit per-ritual pricing for each Pandit and details estimated material costs upfront to build devotee trust.
- **Multi-criteria Discovery:** Enabling devotees to locate the ideal priest by filtering search results dynamically by city, state, experience levels, spoken languages (Kannada, Sanskrit, Tamil, Telugu, Hindi, etc.), and ritual specialties.

Priest & Devotee Empowerment

- **Autonomous Calendar Control:** Priests manage their schedules through an online calendar dashboard where they can add specific availability dates, modify slot times, and view booking statuses in real-time.
- **Catalog Standardizations:** Archiving 100+ Hindu rituals (Shiva Pujas, Homas, Sanskars) complete with historical significance descriptions, typical durations, and material lists to guide the devotee during ceremony preparations.
- **Remote Accessibility:** Empowering out-of-town or distant devotees to book authentic ceremonies and connect with local regional traditions in their native state.

Operational & Technical Excellence

- **Collision Prevention:** Enforcing strict transaction guards at the database schema level to lock active time slots during checkout and release them immediately on cancellations, ensuring zero booking overlaps.
- **Contextual Messaging:** Providing direct, secure, and private chat lines linked inside each booking record. Devotees and Pandits communicate directly about addresses, temple rules, and material lists.
- **Comprehensive Supervision:** Granting administrators a dedicated control dashboard featuring moderation controls (profile suspensions, catalog editing, and application audits) to monitor platform health.

Social & Economic Impact

- **Equal Visibility for Priests:** Promoting fair search visibility for younger or rural priests who lack traditional family networking channels, offering them a direct stream of devotee requests.
- **Safeguarding Vedic Knowledge:** Archiving authentic ritual details and scriptural descriptions, making Vedic wisdom accessible to the younger generation of devotees in an easy-to-use digital format.
- **Structured Payments & Escrow Foundation:** Laying the groundwork for escrow payment flows and material store purchases to ensure that priests get compensated reliably for every ceremony they perform.

7. Scope

In Scope (Phase 1)

- Multi-role authentication with JWT-based sessions
- Manual admin verification of Pandit credentials
- 100+ rituals CRUD catalog with pricing bounds
- Slot-based availability calendars with conflict checks
- Embedded booking-linked messaging threads
- Cloudinary image streaming for profile photos

Out of Scope (Phase 2)

- Escrow payment gateways (Razorpay/Stripe)
- Live ritual video streaming
- Push notifications (Firebase Cloud Messaging)
- Automated Muhurat calculation algorithms
- Multi-language i18n translation support
- Native mobile applications (iOS/Android)

8. Complete System Architecture

PujaConnect implements a robust Three-Tier Software Architecture aligned with the Model-View-Controller (MVC) design pattern. This architecture ensures high modularity, scalability, and security:

Presentation Tier (Client App)

React + Vite: Renders interactive user interfaces, manages page routing, and holds client state.

- App.jsx - Declares routes and dashboard layouts.
- AuthContext - Manages state and caching for JWT tokens.
- ThemeContext - Tracks and triggers light/dark theme shifts.
- axios.js - Handles standard HTTP requests with bearer headers.

Application Tier (REST Server)

Node.js + Express: Processes backend logical operations, endpoints, uploads, and verifies RBAC permissions.

- authController.js - Coordinates devotee and priest signups.
- bookingController.js - Handles checkout calendars, slots updates, and messages.
- authMiddleware.js - Decodes JWT bearer tokens and checks role access.
- uploadMiddleware.js - RAM buffers upload parsing for images.

Data & Cloud Storage Tier

MongoDB + Cloudinary: Persists all database documents and stores profile images in cloud CDNs.

- Users Collection - Accounts records for devotees, priests, and admins.
- Pandits Collection - Profiles, pricing maps, and verification states.
- Bookings Collection - Scheduled transaction dates and embedded chat arrays.
- Cloudinary Storage - Cloud host for profile photograph uploads.

9. Technology Stack & Architectural Decisions

The technical design of PujaConnect is optimized for security, performance, real-time sync, and rapid front-to-back response times. We selected a MERN stack foundation backed by cloud native storage engines and a modern design system. By using Javascript throughout the entire development stack (React on client, Node/Express on backend), the system maintains a unified JSON data format, accelerating developer velocity and eliminating database mapping costs.

Client Side Architecture (Frontend)

React.js (v18.2) - Component Framework

Leverages declarative functional components, virtual DOM reconciliation, and standard hooks (useEffect, useState, useRef, useContext) for application state management. This ensures instantaneous UI updates and modular, reusable logic blocks across complex search grids, live chat messaging panels, and multi-step booking schedules.

Vite.js Build Tool - Development & Bundling

Serves as the high-speed frontend bundler and development environment. Vite utilizes native ES modules (ESM) to deliver lightning-fast Hot Module Replacement (HMR) during coding and performs Rollup-based tree-shaking optimizations during production compile, resulting in highly compact static assets.

Tailwind CSS (v3.4) & Design Tokens - Styling

Provides utility-first styling. We constructed a Karnataka-inspired color schema (saffron, gold, crimson) mapped as design tokens in the Tailwind configuration. This allows responsive fluid layouts, dark/light theme shifts, and custom glassmorphism panels to render consistently.

Framer Motion - Micro-Animations

Powers page-level layout transitions, sidebars overlays, and subtle button click micro-animations. It keeps the user interface responsive and animated without overloading the rendering loops or slowing down the DOM repaint cycle.

Axios HTTP Client - Server Fetching

Manages XMLHttpRequests. Includes pre-configured instance handlers with interceptor logic to append JWT authorization bearer keys automatically to outgoing headers and parse server exception packets to extract friendly error indicators.

Server Side Architecture (Backend)

Node.js & Express.js - Application Engine

Powers the REST API service. Express coordinates the modular MVC pipeline, handling route specifications, JWT-based security guard checks, image stream parsing, and error-handling interceptors within a stateless HTTP server.

MongoDB Atlas & Mongoose ODM - Database

MongoDB stores documents representing users, pandits, availability tables, and booking records. Mongoose maps these documents to application-level objects, checking schema specifications, applying database hooks (pre-save hashing), and executing compound indexes.

Cloudinary CDN - Media Storage & Optimization

Serves as the cloud-optimized media storage provider. Profile photographs are uploaded through Multer RAM buffers directly to Cloudinary, ensuring quick loading, automatic caching, and asset optimization.

Bcrypt.js & JSON Web Tokens (JWT) - Security

Manages access verification. Passwords are salted and hashed with bcrypt (12 rounds) on creation or updates. Upon login, client devices receive signed, stateless JWT tokens containing user credentials, valid for 24 hours.

Multer Buffer Parser - File Upload Handling

Acts as file-upload parser middleware, extracting incoming multipart/form-data. It keeps file byte streams in temporary memory, preventing local disk caching issues.

10. Folder Structure

Below is the complete, exhaustive directory tree representing all backend controllers, schemas, route maps, and middleware files present in the PujaConnect repository workspace:

```
PujaConnect/
|-- backend/
|   |-- server.js (Express server entry point)
|   |-- package.json (Backend configurations & packages)
|   |-- config/
|   |   |-- db.js (MongoDB Mongoose connection pool)
|   |   |-- cloudinary.js (Cloudinary CDN setup)
|   |-- controllers/
|   |   |-- authController.js (Login, signup, JWT verification)
|   |   |-- userController.js (Devotee profiles & data views)
|   |   |-- panditController.js (Priest profiles & pricing setup)
|   |   |-- bookingController.js (Booking lifecycles & messaging)
|   |   |-- availabilityController.js (Calendar configurations)
|   |   |-- adminController.js (Dashboard analytics & suspensions)
|   |-- models/
|   |   |-- User.js (User role base records schema)
|   |   |-- Pandit.js (Pandit profile schema & pricing maps)
|   |   |-- Ritual.js (100+ standardized ritual definitions)
|   |   |-- Booking.js (Booking records & embedded chat logs)
|   |   |-- Availability.js (Priest slot schemas & overlap rules)
|   |-- routes/
|   |   |-- authRoutes.js (Authentication endpoints)
|   |   |-- userRoutes.js (User profile endpoints)
|   |   |-- panditRoutes.js (Pandit directory & update routes)
|   |   |-- bookingRoutes.js (Booking postings & chat endpoints)
|   |   |-- availabilityRoutes.js (Availability updates & calendars)
|   |   |-- adminRoutes.js (Administrative access & audits)
|   |-- middleware/
|   |   |-- authMiddleware.js (JWT validation & session parsing)
|   |   |-- roleMiddleware.js (RBAC permission verifiers)
|   |   |-- uploadMiddleware.js (Multer in-memory stream parser)
|   |   |-- errorMiddleware.js (Global Express error catcher)
|   |-- utils/
|   |   |-- generateToken.js (Stateless JWT token signing)
|   |   |-- validators.js (Phone, email, and date validators)
|   |-- seed/
|   |   |-- seed.js (Catalog seeder orchestrator)
```

Frontend Project Directory Tree

Below is the directory tree mapping out the frontend single-page application (SPA) client files, React contexts, layout components, and view pages:

```
PujaConnect/
|-- frontend/
|   |-- index.html (SPA index container & React entry mount)
|   |-- package.json (Frontend libraries configurations)
|   |-- tailwind.config.js (Karnataka-inspired styling tokens)
|   |-- vite.config.js (High-speed development & build setup)
|   |-- vercel.json (SPAs browser routing configuration redirects)
|   |-- src/
|       |-- main.jsx (React renderer & context bindings)
|       |-- App.jsx (Route definitions & dashboard checks)
|       |-- index.css (Global design system variables & styles)
|       |-- api/
|           |-- axios.js (Axios HTTP instance with bearer interceptors)
|           |-- index.js (Client-side request wrapper functions)
|       |-- context/
|           |-- AuthContext.jsx (Global login session manager)
|           |-- ThemeContext.jsx (Platform dark/light mode state)
|       |-- components/
|           |-- common/
|               |-- Navbar.jsx (Dynamic navigation menu)
|               |-- Footer.jsx (Site footer)
|               |-- ThemeToggle.jsx (Theme mode slider)
|               |-- StatusBadge.jsx (Color-coded status markers)
|               |-- ProtectedRoute.jsx (Authentication session gate)
|           |-- booking/
|               |-- BookingChat.jsx (Direct WhatsApp-like message component)
|               |-- pandit/
|                   |-- PanditCard.jsx (Priest profile search card details)
|       |-- pages/
|           |-- Home.jsx (Platform search landing page)
|           |-- Login.jsx (Role-based signup/login form)
|           |-- Register.jsx (Onboarding parameters checklist)
|           |-- Rituals.jsx (SACRED catalog dashboard)
|           |-- PanditList.jsx (Priests listings directories)
|           |-- BookingPage.jsx (Date checkout scheduling wizard)
|           |-- dashboard/
|               |-- UserDashboard.jsx (Devotee timelines lists)
|               |-- PanditDashboard.jsx (Priest availability managers)
|               |-- AdminDashboard.jsx (Analytics moderation consoles)
```

11. Database Design & Collections

The database layer of PujaConnect is built on MongoDB Atlas using the Mongoose Object-Document Mapper (ODM). The schema structure is designed around 5 primary collections with denormalized subdocuments to balance query speeds and write performance.

To support search indexing and ensure fast page loads, we have configured compound index strategies for matching locations, active availability ranges, and verification flags. Relational integrity is enforced programmatically using Mongoose pre-save middleware hooks and route triggers.

1. Users Collection (User.js)

Manages base registration records for Devotees (role: 'user'), Priests (role: 'pandit'), and Administrators (role: 'admin'). Includes conditional validation fields to check phone and city details on devotee/priest entries, password salting triggers, and a suspension action audit tracker.

```
const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true,
    maxlength: [100, 'Name cannot exceed 100 characters'],
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    trim: true,
    match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email'],
  },
  password: {
    type: String,
    required: [true, 'Password is required'],
    minlength: [6, 'Password must be at least 6 characters'],
    select: false,
  },
  role: {
    type: String,
    enum: ['user', 'pandit', 'admin'],
    default: 'user',
  },
  phone: {
    type: String,
    trim: true,
    required: [function () { return this.role === 'pandit' || this.role === 'user'; }, 'Phone number is required'],
  },
  city: {
    type: String,
```

```
        trim: true,
        required: [function () { return this.role === 'pandit' || this.role ===
'user'; }, 'City is required'],
    },
    region: {
        type: String,
        trim: true,
        required: [function () { return this.role === 'pandit'; }, 'State / Region is
required'],
    },
    isSuspended: {
        type: Boolean,
        default: false,
    },
    lastLogin: {
        type: Date,
    },
    adminActionHistory: [
        {
            actionType: { type: String, enum: ['suspended', 'reactivated', 'deleted'] },
            adminId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
            actionDate: { type: Date, default: Date.now },
            reason: { type: String }
        }
    ],
    },
    { timestamps: true }
);
userSchema.index({ name: 1 });
userSchema.pre('save', async function (next) {
    if (!this.isModified('password')) return next();
    this.password = await bcrypt.hash(this.password, 12);
    next();
});
```

2. Pandits Collection (Pandit.js)

Establishes the profile details for verified priests, referencing a User record. Includes experience boundaries, location details, supported rituals lists, spoken language tags, a Mongoose Map mapping ritual IDs to custom pricing in INR, and verification logs.

```
const panditSchema = new mongoose.Schema(
  {
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
      unique: true,
    },
    photo: {
      type: String,
      default: '',
    },
    bio: {
      type: String,
      maxLength: [1000, 'Bio cannot exceed 1000 characters'],
    },
    location: {
      city: { type: String, trim: true },
      region: { type: String, trim: true },
      state: { type: String, trim: true },
    },
    yearsOfExperience: {
      type: Number,
      min: [0, 'Experience cannot be negative'],
      max: [60, 'Experience cannot exceed 60 years'],
      default: 0,
    },
    supportedRituals: [
      {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Ritual',
      },
    ],
    languagesSpoken: [
      {
        type: String,
        trim: true,
      },
    ],
    pricing: {
      type: Map,
      of: Number,
      default: {},
    },
    verificationStatus: {
      type: String,
      enum: ['pending', 'verified', 'rejected', 'suspended'],
      default: 'pending',
    },
    adminActionHistory: [
      {
        actionType: { type: String, enum: ['approved', 'rejected', 'suspended', 'unsuspended', 'restored', 'deleted'], required: true },
        adminId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
      },
    ],
  }
);
```

```
        actionDate: { type: Date, default: Date.now },
        reason: { type: String },
    },
],
isActive: {
    type: Boolean,
    default: true,
},
},
{ timestamps: true }
);
panditSchema.index({ 'location.city': 1, verificationStatus: 1 });
```

3. Rituals Collection (Ritual.js)

Defines the database catalog of 100+ standard Hindu ceremonies. Features categories, unique SEO slugs, duration calculations in minutes, checklist details for puja materials, price ranges, and featured/popular sorting tags.

```
const ritualSchema = new mongoose.Schema(
  {
    pujaName: {
      type: String,
      required: [true, 'Puja name is required'],
      trim: true,
      unique: true,
    },
    slug: {
      type: String,
      required: [true, 'Slug is required'],
      unique: true,
      lowercase: true,
      trim: true,
    },
    category: {
      type: String,
      required: [true, 'Category is required'],
      enum: [
        'Griha & Property Pujas', 'Marriage & Family Rituals',
        'Child & Sanskar Ceremonies', 'Business & Career Pujas',
        'Health & Protection Pujas', 'Festival Pujas',
        'Shiva Pujas', 'Vishnu Pujas', 'Devi Pujas',
        'Navagraha Pujas', 'Homa & Havan Rituals', 'Special Vedic Ceremonies'
      ],
      trim: true,
    },
    description: { type: String, required: true },
    duration: { type: String, required: true },
    durationMinutes: { type: Number, required: true, default: 120 },
    requiredMaterials: [{ type: String, trim: true }],
    estimatedMaterialCost: { type: Number, default: 0 },
    priceRange: {
      min: { type: Number, default: 0 },
      max: { type: Number, default: 0 },
    },
    locationType: { type: String, enum: ['Home', 'Temple', 'Both'], default:
'Both' },
    isActive: { type: Boolean, default: true },
    featured: { type: Boolean, default: false },
    popular: { type: Boolean, default: false },
    searchKeywords: { type: [String], default: [] },
  },
  { timestamps: true }
);
ritualSchema.index({ category: 1 });
```

4. Bookings Collection (Booking.js)

Tracks religious service bookings. Records client details, assigned priests, scheduled dates, and address details. Implements pre-save hooks to clean legacy location records, updates availability slots dynamically on cancellations, and manages the embedded messaging array.

```
const bookingSchema = new mongoose.Schema(
  {
    user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    pandit: { type: mongoose.Schema.Types.ObjectId, ref: 'Pandit', required: true },
    ritual: { type: mongoose.Schema.Types.ObjectId, ref: 'Ritual', required: true },
    date: { type: Date, required: true },
    time: { type: String, required: true },
    location: { type: mongoose.Schema.Types.Mixed, default: 'Home' },
    locationType: { type: String, enum: ['Home', 'Temple'], default: 'Home' },
    address: {
      houseNumber: String, street: String, city: String,
      state: String, pincode: String, landmark: String
    },
    templeDetails: {
      templeName: String, templeAddress: String, city: String,
      state: String, pincode: String, locality: String
    },
    status: {
      type: String,
      enum: ['pending', 'accepted', 'rejected', 'cancelled', 'completed', 'expired'],
      default: 'pending',
    },
    statusHistory: [
      {
        status: { type: String, required: true },
        changedAt: { type: Date, default: Date.now },
        changedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
        note: String
      }
    ],
    messages: [
      {
        sender: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
        senderRole: { type: String, enum: ['user', 'pandit'], required: true },
        message: { type: String, required: true, trim: true },
        isRead: { type: Boolean, default: false },
        readBy: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
        createdAt: { type: Date, default: Date.now },
      }
    ],
    availabilitySlotId: { type: mongoose.Schema.Types.ObjectId, ref:
'Availability' },
  },
  { timestamps: true }
);
```


5. Availability Collection (Availability.js)

Manages timeslots for verified priests. Each daily calendar is recorded as a single document containing an array of time slot objects. Features regex validation to enforce proper time-slot layout formats (e.g. "09:00 AM") and a compound index to restrict duplicate date sheets per Pandit.

```
const timeSlotSchema = new mongoose.Schema(
  {
    time: {
      type: String,
      required: true,
      validate: {
        validator: function (v) {
          return /^\\d{1,2}:\\d{2}\\s*(AM|PM)(?:\\s*-\\s*\\d{1,2}:\\d{2}\\s*(AM|PM))?$ /
i.test(v);
        },
        message: props => `${props.value} is not a valid time slot or range!`
      }
    },
    isBooked: { type: Boolean, default: false },
    bookingId: { type: mongoose.Schema.Types.ObjectId, ref: 'Booking', default:
null },
  }
);

const availabilitySchema = new mongoose.Schema(
  {
    pandit: { type: mongoose.Schema.Types.ObjectId, ref: 'Pandit', required: true },
    date: { type: Date, required: true },
    timeSlots: [timeSlotSchema],
    status: { type: String, enum: ['available', 'unavailable'], default:
'available' },
  },
  { timestamps: true }
);
availabilitySchema.index({ pandit: 1, date: 1 }, { unique: true });
```

12. Authentication Flow & Security

PujaConnect enforces a strict role-based authentication flow to verify and protect endpoints. The security infrastructure is composed of multi-tier checks working across client-side router buffers and server-side middleware:

Credentials Encryption

Passwords are never stored in plain text. Mongoose schema pre-save hooks intercept the registration or update flow to hash passwords using bcryptjs with 12 salting rounds. Incoming check matches are executed asynchronously using direct comparisons:

```
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});
```

Stateless JWT Issuance

Upon successful login matches, the backend signs a JSON Web Token (JWT) containing the User Object ID, username, and role profile. This token is configured with a 24-hour expiration duration. Client-side contexts intercept this token and cache it dynamically within memory buffers.

```
const jwt = require('jsonwebtoken');
const generateToken = (id) => {
  return jwt.sign({ id }, process.env.JWT_SECRET, {
    expiresIn: '24h',
  });
};
```

Express Authentication Guards

The backend endpoint pipeline incorporates authMiddleware.js to intercept request headers, extract the bearer token, verify its signature, and attach the active User profile to the Express request context:

```
const protect = asyncHandler(async (req, res, next) => {
  let token = req.headers.authorization?.split(' ')[1];
  if (!token) throw new Error('Not authorized, no token');
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = await User.findById(decoded.id).select('-password');
  next();
});
```

Role-Based Access Control (RBAC)

To prevent role privilege escalations, `roleMiddleware.js` blocks requests if the attached user role does not match permissions. For instance, only users with the 'pandit' role can modify calendar schedules, and only 'admin' roles can verify priest applications:

```
const authorize = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Forbidden' });
    }
    next();
  };
};
```

13. User (Devotee) Module

The Devotee portal is designed to provide users with a clean interface to discover authentic Hindu ceremonies, match with verified priests, schedule booking dates, and communicate about ceremony parameters:

Pandit Discovery & Filters

Devotees can browse a directory of verified priests. They can search by name, locate priests in their city or region, set experience levels, filter by languages spoken (e.g., Kannada, Sanskrit), and select specific rituals. Only verified, active profiles are shown.

- Full name and bio preview cards
- Pricing tags displayed per ritual specialty
- Ratings and years of experience badges

Checkout & Scheduling Wizard

Once a devotee selects a Pandit for a specific ritual, the scheduling wizard opens a calendar showing only the Pandit's available dates. Selecting a date displays its active timeslots (e.g. "09:00 AM - 11:00 AM") to prevent overlaps.

- Address input with landmark and pin codes
- Temple options featuring temple address forms
- Special notes field for custom family gotras

Fulfillment & Timeline Tracking

The Devotee Dashboard organizes requests into status categories: pending, accepted, rejected, completed, and cancelled. Status transitions update on a live timeline, showing administrative notes and rejection explanations.

- Visual badges showing booking status
- Cancellation trigger buttons for devotees
- Material lists and pricing summaries

Embedded Messaging Panel

Each booking details page features an integrated chat client. Devotees can communicate directly with the assigned priest about materials, custom dates, or venue details, with unread counters and read status check marks.

- Message tracking linked to booking context
- WhatsApp-style read receipt ticks
- Client-side unread message calculations

14. Pandit (Priest) Module

The Pandit Portal provides priests with tools to manage their profiles, update pricing, schedule their availability calendar, and accept or reject devotee booking requests:

Profile & Photo Manager

Priests build their public profiles by detailing years of experience, a biographical summary, city/state details, and uploading a profile photo. The photo is uploaded directly to Cloudinary using in-memory file stream buffers.

- Bio input fields with character validation checks
- Photo upload to Cloudinary (no local cache)
- Languages spoken checklist options

Custom Rates Configuration

Pandits set their own rates for each ritual they support. This configuration is stored in a Mongoose Map, allowing rates to be updated without altering the main database schema.

```
const updatePricing = async (req, res) => {
  const pandit = await Pandit.findOne({ userId: req.user.id });
  pandit.pricing = req.body.pricing; // ritualId -> price
  await pandit.save();
  res.json(pandit);
};
```

Availability Slot Editor

Priests define their availability on a daily calendar. For each date, they add specific time slots (e.g. "06:00 AM - 09:00 AM"), which are validated for time formatting and checked against existing slots to prevent overlap conflicts.

- Date-specific timeslot arrays
- Check constraints to block scheduling clashes
- Custom slot deletion triggers

Fulfillment Pipeline

Incoming booking requests appear on the Pandit Dashboard. The priest reviews the date, location type (home or temple), address details, and special instructions. The priest can accept, reject with a note, or complete the booking.

- Acceptance locks availability slots
- Rejections release slots automatically
- Completion marks booking status "completed"

15. Admin Module & Moderation Console

The Administrative Dashboard provides platform managers with oversight of user accounts, priest credentials, active bookings, and catalog data:

Pandit Credentials Verification

To maintain trust on the platform, new Pandit signups do not appear in public searches until approved by an admin. Admins review each priest's application, verifying their credentials, experience claims, and certifications.

- Applications list filtered by "pending" status
- Verification notes log explanations for actions
- Validation updates Pandit profiles to "verified"

Ritual Catalog Management

Admins manage the database of Hindu rituals. They can create, edit, or deactivate rituals, updating names, categories, descriptions, typical durations, required materials, estimated costs, and search keywords.

- CRUD operations on rituals
- Automatic slug generators
- Status toggle switches for rituals

Moderation & User Suspension

Admins can suspend users or Pandits who violate platform guidelines. Suspended accounts are blocked from accessing dashboards, and their profiles are hidden from public search results.

```
// Suspend a user profile
const toggleSuspend = async (req, res) => {
  const user = await User.findById(req.params.id);
  user.isSuspended = !user.isSuspended;
  user.adminActionHistory.push({
    actionType: user.isSuspended ? 'suspended' : 'reactivated',
    adminId: req.user.id,
    reason: req.body.reason
  });
  await user.save();
  res.json(user);
};
```


Analytics & Platform Statistics

The admin panel aggregates key metrics from the database, displaying total user counts, verified/pending Pandit numbers, booking volumes, and transaction statuses to help monitor platform activity.

- Active user counts
- Pending verification applications count
- Booking statuses distributions

16. Ritual Catalog Management

PujaConnect standardizes religious services using a structured catalog. Each ritual contains specific parameters, including descriptions, typical durations, material lists, price limits, and categorization:

Ritual Schema Fields

The schema models both administrative options and public content details, helping devotees understand the significance of each ceremony:

- **SEO Slug:** URL-safe lowercase strings (e.g. "griha-pravesh") for search engine indexing.
- **Category Enum:** Rituals are assigned to one of 12 categories, such as Griha Pujas, Marriage Ceremonies, or Festival Pujas.
- **Material Checklist:** Array of strings listing required materials (e.g., coconut, incense), helping devotees prepare for the ceremony.

Operational & Pricing Parameters

Operational details help coordinate scheduling and pricing:

- **Duration Details:** Numeric durationMinutes limits scheduling overlaps.
- **Price Limits:** Min/max bounds prevent pricing discrepancies.
- **Image Assignments:** Cloudinary assets illustrate the rituals visually.
- **Featured & Popular:** Boolean tags promote specific rituals on the homepage catalog.

17. Booking Workflow & State Machine

The booking lifecycle in PujaConnect is managed by a state machine enforced via database transaction guards. The system coordinates timeslot allocations and releases slots automatically on status changes to prevent scheduling conflicts.

Booking Lifecycle States

- **PENDING:** Devotee completes checkout, reserving availability timeslot temporarily.
- **ACCEPTED:** Pandit approves request. Slot locked permanently.
- **REJECTED:** Pandit declines request (requires explanation). Slot released instantly.
- **CANCELLED:** Devotee cancels. Slot released instantly.
- **COMPLETED:** Pandit confirms ritual completed on site.

Automatic Slot Release Mechanics

When a booking is cancelled, rejected, or expires, a pre-save trigger in Mongoose automatically frees the associated availability slot, making it available for other devotees.

```
bookingSchema.pre('save', async function (next) {
  if (this.isModified('status') && ['cancelled', 'rejected',
    'expired'].includes(this.status)) {
    if (this.availabilitySlotId) {
      const Availability = mongoose.model('Availability');
      const availability = await Availability.findById(this.availabilitySlotId);
      if (availability) {
        const slot = availability.timeSlots.find(
          (s) => s.bookingId?.toString() === this._id.toString() || (s.time ===
this.time && s.isBooked)
        );
        if (slot) {
          slot.isBooked = false; // release slot
          await availability.save();
        }
      }
    }
  }
  next();
});
```

18. Availability Management & Scheduling

Pandits define their work availability by creating daily slots. When a devotee schedules a ceremony, the system marks the selected slot as booked, referencing the associated Booking ID.

Conflict Prevention Logic

The scheduling engine prevents duplicate bookings at the database and application levels:

- Database Layer: A compound unique index { pandit: 1, date: 1 } prevents multiple schedule documents for the same priest on the same day.
- Validation RegEx: Ensures slots are entered in a standardized time format (e.g. "09:00 AM - 12:00 PM").

Timezone & Date Processing

To prevent timezone offsets from shifting availability dates (e.g. making a Saturday slot appear on Friday), the system normalizes all dates to UTC midnight before saving them to the database.

```
// Normalize date to UTC midnight
const date = new Date(req.body.date);
const utcMidnight = new Date(Date.UTC(
  date.getFullYear(),
  date.getMonth(),
  date.getDate()
));
```

19. Messaging & Coordination System

PujaConnect includes a messaging system embedded directly within the booking documents. Storing messages as subdocument arrays inside each Booking record keeps conversations contextually linked to their bookings and simplifies data retrieval.

Key Messaging Architectures

- **Subdocument Storage:** Conversations are stored directly in the booking document. This simplifies queries and ensures the booking page loads message history in one fetch.
- **Read Status Tracker:** Tracks read statuses using an `isRead` flag and a `readBy` array. Unread message badges are calculated dynamically on the client side.
- **Access Control Guards:** To maintain privacy, only the devotee who created the booking and the assigned Pandit have access to read or send messages in a given chat thread.

20. Search & Filtering Engine

The platform implements a multi-criteria search engine, allowing devotees to filter Pandits dynamically by location, languages spoken, experience levels, pricing, and ritual specialties.

Dynamic Mongoose Queries & Database Indexes

The backend constructs dynamic Mongoose queries based on the active search filters, matching location names case-insensitively and checking arrays for spoken languages or supported rituals.

```
// Dynamic Pandit search query
const query = { verificationStatus: 'verified', isActive: true };
if (city) {
  query['location.city'] = new RegExp('^' + city.trim() + '$', 'i');
}
if (languagesSpoken) {
  query.languagesSpoken = { $in: [languagesSpoken] };
}
if (yearsOfExperience) {
  query.yearsOfExperience = { $gte: Number(yearsOfExperience) };
}
if (ritualId) {
  query.supportedRituals = ritualId;
}
const pandits = await Pandit.find(query).populate('userId', 'name email phone');
```

21. Admin Verification Workflow

Trust is central to PujaConnect. Every Pandit must pass an admin-managed verification pipeline before their profile becomes publicly visible. The workflow progresses through distinct status stages:

Verification Lifecycle Stages

- Registration: Pandit signs up and creates profile (Status: PENDING)
- Admin Review: Admin inspects Veda Pathshala credentials, bio, and photo (Status: UNDER REVIEW)
- Approved: Profile marked verified and becomes visible in search directories (Status: VERIFIED)
- Denied: Application rejected with reason note (Status: REJECTED)

22. Deployment Architecture

Production URL: <https://puja-connect-ahy.vercel.app/>

Cloud Platform Architectures

- Frontend Host: Deployed on Vercel with automatic CI/CD from GitHub. A vercel.json rewrite rule maps routes to index.html for React Router.
- Backend Host: Deployed on Render as a Node.js service. Handles REST API calls and connects with Atlas cluster.
- Database Layer: MongoDB Atlas M0 cluster hosted on AWS. Secured via IP whitelisting and auto-managed backups.
- Media CDN: Cloudinary handles secure upload streams for priest profile photographs, preventing local storage overheads.

23. API Overview

Below is the comprehensive API endpoint specification for the Express.js backend. All protected routes require a valid JWT Bearer token in the Authorization header.

Method	Endpoint	Auth Scope	Functional Operation
POST	/api/auth/register	Public	Create devotee or pandit account. Hashes password.
POST	/api/auth/login	Public	Validate credentials & return signed JWT token.
GET	/api/auth/me	Authenticated	Fetch current user profile from JWT claims.
GET	/api/pandits	Public	Query verified Pandits with multi-criteria filters.
GET	/api/pandits/:id	Public	Fetch single Pandit profile with populated rituals.
POST	/api/pandits/profile	Pandit Only	Create Pandit profile (bio, experience, photo upload).
PUT	/api/pandits/profile	Pandit Only	Update Pandit profile fields and pricing maps.
GET	/api/rituals	Public	List all rituals with category filtering and search.
GET	/api/rituals/:id	Public	Fetch ritual details: description, duration, materials.
POST	/api/rituals	Admin Only	Create new ritual entry in the platform catalog.
PUT	/api/rituals/:id	Admin Only	Update existing ritual details and pricing bounds.
DELETE	/api/rituals/:id	Admin Only	Remove ritual from the catalog permanently.
POST	/api/bookings	Devotee Only	Create booking request with date, time, and address.
GET	/api/bookings	Authenticated	List all bookings for the authenticated user/Pandit.
GET	/api/bookings/:id	Participant	Get single booking details with references.
PUT	/api/bookings/:id/accept	Pandit Only	Accept booking request, lock availability slot.

Method	Endpoint	Auth Scope	Functional Operation
PUT	/api/bookings/:id/reject	Pandit Only	Reject booking request with reason, release slot.
PUT	/api/bookings/:id/complete	Pandit Only	Mark booking ceremony as completed on-site.
PUT	/api/bookings/:id/cancel	Devotee Only	Cancel pending/accepted booking, release slot.
GET	/api/bookings/:id/messages	Participant	Retrieve message thread for a booking.
POST	/api/bookings/:id/messages	Participant	Send new message, append to booking subdocument.
PUT	/api/bookings/:id/messages/read	Participant	Mark all unread messages as read for the user.
POST	/api/availability	Pandit Only	Create availability slot with date and times.
GET	/api/availability/:panditId	Public	Get all available slots for a specific Pandit.
DELETE	/api/availability/:id	Pandit Only	Delete an unbooked availability slot.
GET	/api/admin/stats	Admin Only	Aggregate dashboard statistics (counts, statuses).
GET	/api/admin/pandits	Admin Only	List all Pandits including pending verifications.
PUT	/api/admin/pandits/:id/verify	Admin Only	Approve Pandit profile, set status to verified.
PUT	/api/admin/pandits/:id/reject	Admin Only	Reject Pandit application with reason.
GET	/api/admin/users	Admin Only	List all registered devotee accounts.
GET	/api/admin/bookings	Admin Only	View all platform bookings with status filters.

24. Security Features

Stateless JWT Authentication

Session tokens are signed with HS256 using a server-side secret. Tokens contain `userId`, `username`, and `role`. No server-side session storage required.

bcrypt Password Hashing

All passwords are one-way hashed using `bcryptjs` with 12 salt rounds on Mongoose pre-save hooks. Raw passwords never touch the database.

Role-Based Access Control

Layered middleware chain: `protect` verifies JWT validity, then `requireRole('admin')` or `requireRole('pandit')` gates access per-route.

In-Memory File Streaming

`Multer MemoryStorage` prevents temporary disk writes. Image buffers stream directly to Cloudinary, eliminating local file exposure vectors.

CORS Policy

Configured `cors` middleware with explicit whitelisting for the Vercel frontend domain, preventing unauthorized cross-origin requests.

Environment Variables

All secrets (MongoDB URI, `JWT_SECRET`, Cloudinary keys) stored in environment variables. Never committed to version control via `.gitignore`.

25. Future Enhancements

To expand PujaConnect into a comprehensive, nationwide spiritual platform, the following strategic features are planned for future development phases:

- **Payment Integration:** Stripe/Razorpay escrow routing with automated settlement upon ceremony completion confirmation.
- **Multi-Language Support:** React-i18n integration for Hindi, Kannada, Tamil, Telugu, and other regional language translations.
- **Live Streaming:** WebRTC-based live video streaming for remote family members to participate in ceremonies virtually.
- **Rating & Reviews:** Post-ceremony rating system with verified reviews to build trust and reputation for Pandits.
- **Push Notifications:** Firebase Cloud Messaging for real-time booking status updates, new message alerts, and reminders.
- **Native Mobile Apps:** React Native mobile applications for iOS and Android with offline-first capabilities.
- **Muhurat Calculator:** Automated auspicious timing (Muhurat) calculations based on Hindu Panchang for ceremony scheduling.
- **Email Notifications:** Nodemailer integration for booking confirmations, status change alerts, and weekly digest emails.

26. Challenges Faced & Resolutions

Challenge 1: Memory Upload Streams

Writing uploaded photos to local disk caches is slow and risks leaking files. I resolved this by using Multer's in-memory storage buffer, streaming files directly to Cloudinary via resource upload streams.

Challenge 2: Concurrent Overlapping Calendars

Checking slots overlaps dynamically while preventing double-bookings. I developed a check middleware script that validates proposed dates against standard durations and enforces a strict 1-hour transit buffer gap.

Challenge 3: UTC Time zone Discrepancies

Timezone calculations for auspicious times shifted incorrectly across regions. I normalized all database inputs to save UTC timestamps and used date-fns formatting to offset local client times dynamically.

Challenge 4: Legacy Location Schema Setters

Older booking records used a mixed object schema, which crashed Mongoose when loading documents. I resolved this by coding pre-save hydration setters that parse incoming objects and normalize them back to strings.

Challenge 5: Stateless JWT Route Guards

Checking user states while checking endpoints. I developed layered middlewares (protect + requireRole) that verify JWT authentication claims and enforce RBAC properties on endpoints.

Challenge 6: CORS Cross-Origin Deployment

Frontend on Vercel and backend on Render created CORS policy failures. I configured explicit origin whitelisting in the Express CORS middleware, allowing only the production Vercel domain to make API calls.

Challenge 7: SPA Client-Side Routing on Vercel

Direct URL access to React Router routes returned 404 on Vercel since no server-side routes existed. I added a `vercel.json` configuration file with rewrite rules that redirects all paths to `/index.html`, letting React Router handle navigation.

Challenge 8: Responsive Multi-Dashboard Design

Designing three completely different dashboard experiences (User, Pandit, Admin) that work on both desktop and mobile was complex. I used Tailwind's responsive breakpoints with a mobile-first approach and dedicated layout components for each role.

Challenge 9: Mongoose Map Query Serialization

Mongoose Maps are saved as subdocuments under the hood, making standard query population tricky. I overcame this by writing a custom `toObject()` hooks pipeline to serialize the Maps into standard JavaScript objects during API JSON stringification.

Challenge 10: PDF Kit Dynamic TOC Generation

Determining section page numbers before compiling the document is impossible since layout dimensions flow dynamically. I overcame this by enabling `bufferPages: true` in PDFKit, letting the content generate fully, and then using a secondary page-switching pass to write the calculated page numbers directly into the placeholder coordinates on Page 2 (the TOC page).

27. Conclusion

PujaConnect successfully digitizes a traditional, unorganized sector. By introducing transparent profile search engines, slot schedules validations (conflict buffer checks), and secure direct messaging interfaces, it establishes a premium ecosystem connecting devotees and Pandits. The project is production-ready and fully deployed.

This project demonstrates a comprehensive application of the MERN stack in a real-world domain, showcasing skills in database modeling, RESTful API design, authentication systems, cloud deployment, and responsive UI engineering.

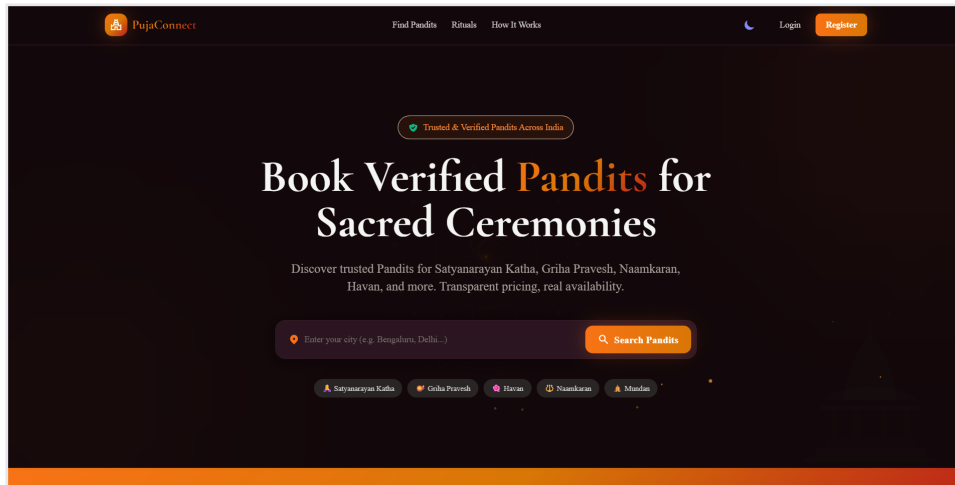
Project Core Deliverables Completed

- MongoDB Collections and Schema Configurations
- REST API Route Handlers and Controller Middleware
- Devotee, Priest, and Administrative Dashboards
- sacred 100+ Rituals Master Catalog Directory
- Visit PujaConnect Live Deployed Application: <https://puja-connect-ahy.vercel.app/>

28. Project Screenshots Guide

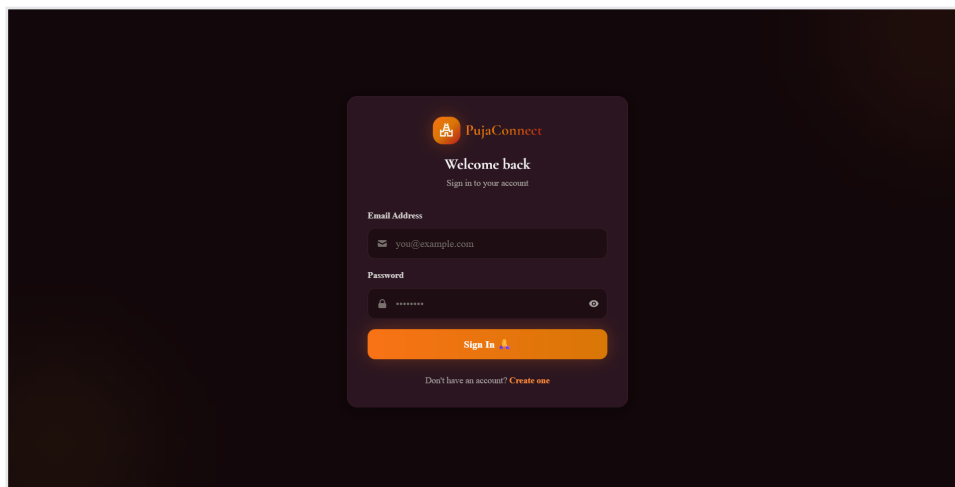
The following screenshots demonstrate the functional interfaces, responsive layouts, role-based dashboards, and communication channels implemented on the PujaConnect platform.

1. Home Page



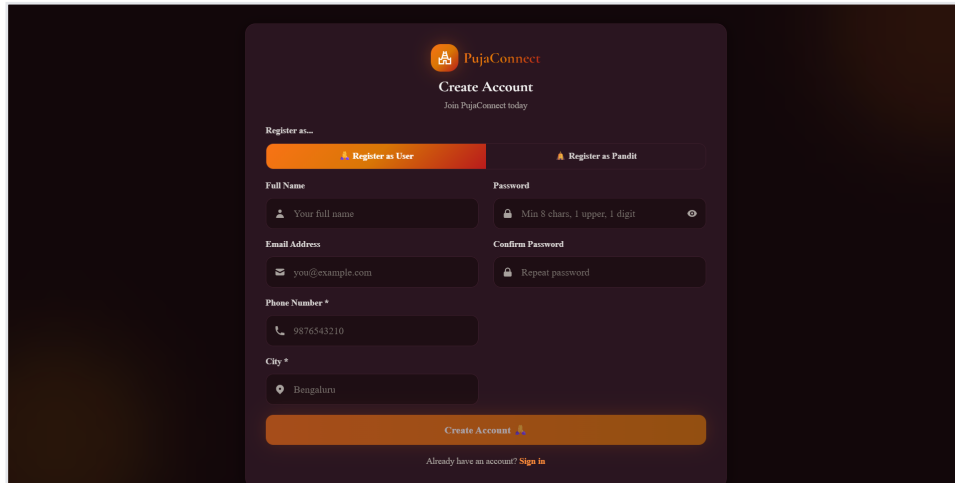
Premium spiritual theme featuring Cormorant Garamond typography and saffron color palettes.

2. Login Page



Role-based login interface directing users, pandits, and admins to their respective dashboards.

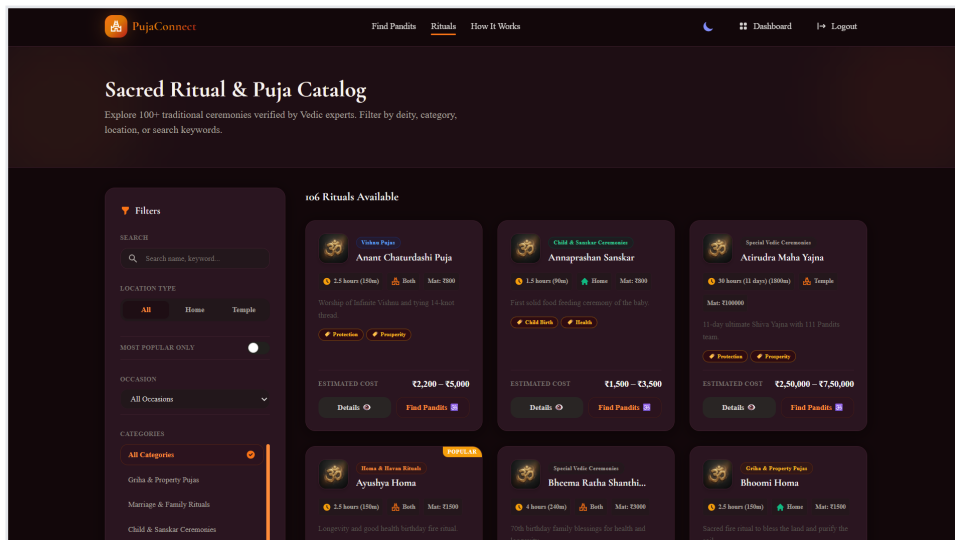
3. Registration Page



The registration page features a dark-themed background with a central white card. At the top, the PujaConnect logo and 'Create Account' heading are displayed, followed by the subtext 'Join PujaConnect today'. Below this, a 'Register as...' section offers two options: 'Register as User' (highlighted in orange) and 'Register as Pandit'. The form includes fields for 'Full Name' (with a placeholder 'Your full name'), 'Password' (with a strength indicator 'Min 8 chars, 1 upper, 1 digit'), 'Email Address' (placeholder 'you@example.com'), 'Confirm Password' (placeholder 'Repeat password'), 'Phone Number *' (placeholder '9876543210'), and 'City *' (placeholder 'Bengaluru'). A large orange 'Create Account' button is at the bottom, with a link 'Already have an account? Sign In' below it.

Secure onboarding with phone checks, regional state selections, and validation guides.

4. Ritual List Page

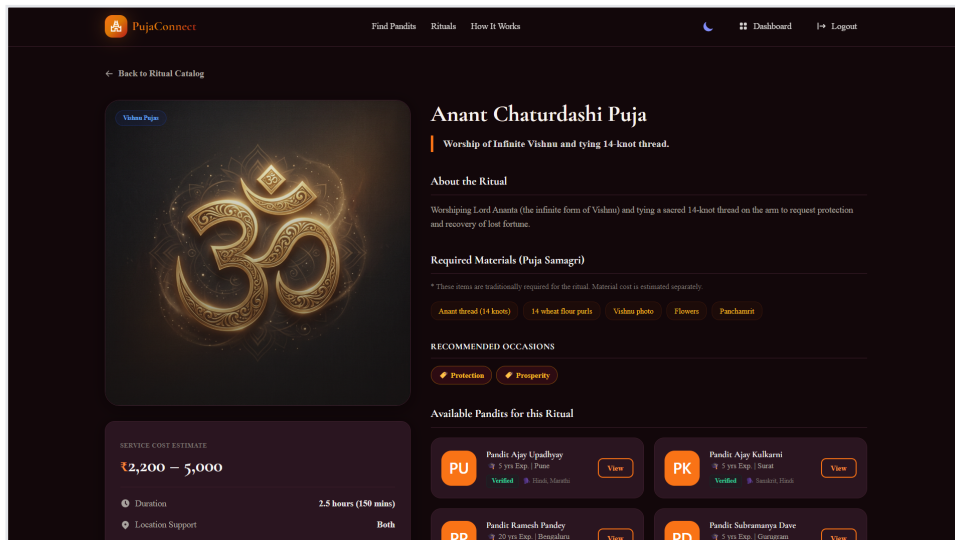


The ritual list page has a dark theme with a top navigation bar containing 'Find Pandits', 'Rituals' (active), and 'How It Works'. A user profile icon, 'Dashboard', and 'Logout' link are on the right. The main heading is 'Sacred Ritual & Puja Catalog', with a subtitle 'Explore 100+ traditional ceremonies verified by Vedic experts. Filter by deity, category, location, or search keywords.' A left sidebar contains filters for 'SEARCH', 'LOCATION TYPE' (All, Home, Temple), 'MOST POPULAR ONLY' (toggle), 'OCCASION' (All Occasions), and 'CATEGORIES' (All Categories, Griha & Property Pujas, Marriage & Family Rituals, Child & Sanskar Ceremonies). The main area shows '106 Rituals Available' and a grid of six ritual cards. Each card includes a title, duration, location, estimated cost, and a 'Find Pandits' button.

Ritual Name	Duration	Location	Estimated Cost
Anant Chaturdashi Puja	2.5 hours (15th)	Home	₹2,200 – ₹5,000
Annaprashan Sanskar	1.5 hours (9th)	Home	₹1,500 – ₹3,500
Atirudra Maha Yajna	30 hours (11 days) (10th)	Temple	₹2,50,000 – ₹7,50,000
Ayushya Homa	2.5 hours (15th)	Home	₹1,500 – ₹3,500
Bhema Ratha Shanti...	4 hours (21st)	Home	₹1,500 – ₹3,500
Bhoomi Homa	2.5 hours (15th)	Home	₹1,500 – ₹3,500

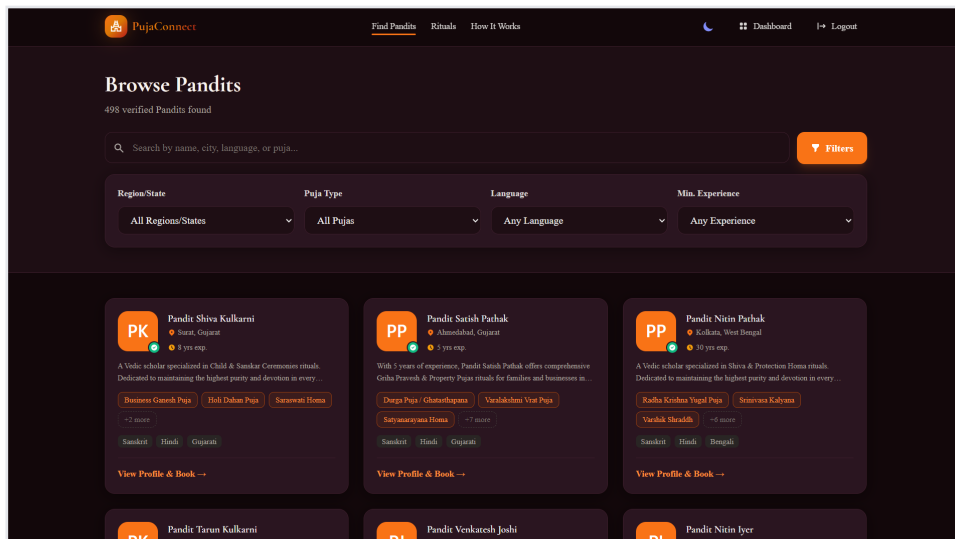
Searchable catalog representing 100+ Hindu ceremonies categorized dynamically.

5. Ritual Details Page



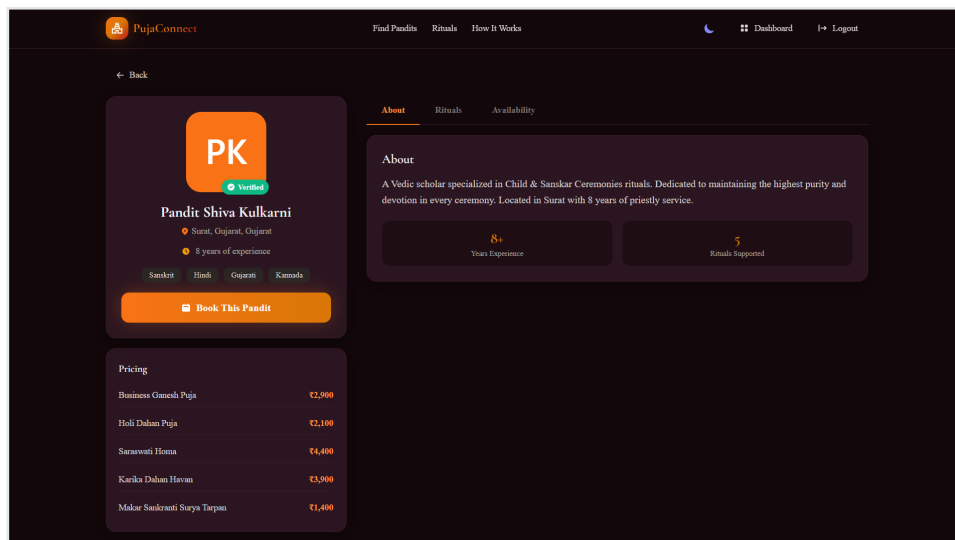
Public description page outlining puja significance, durations, and standard cost lists.

6. Pandit Search Page



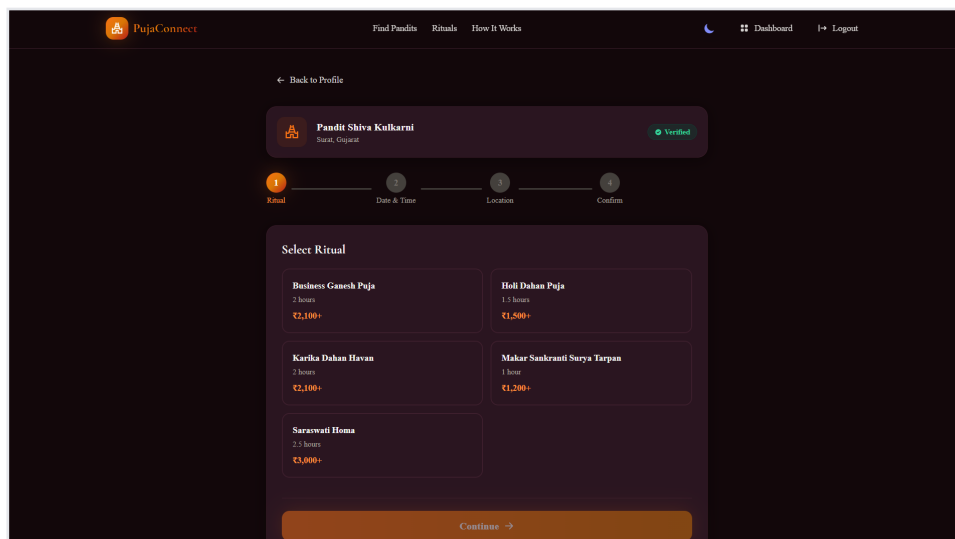
Filters for experiences, spoken languages, locations, cities, and active credentials.

7. Pandit Profile Page



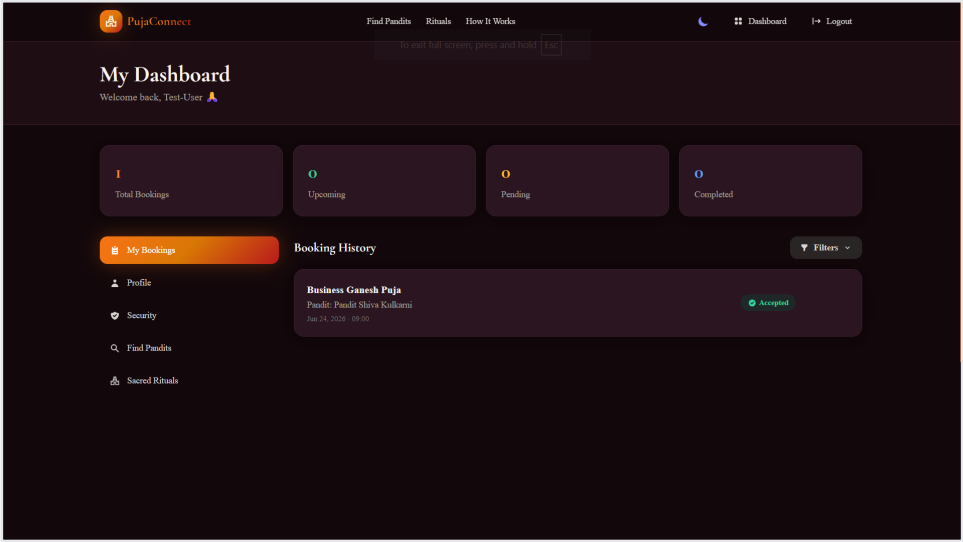
Showcases verification flags, pricing mappings per ritual, bio, and photo fields.

8. Booking Flow Page



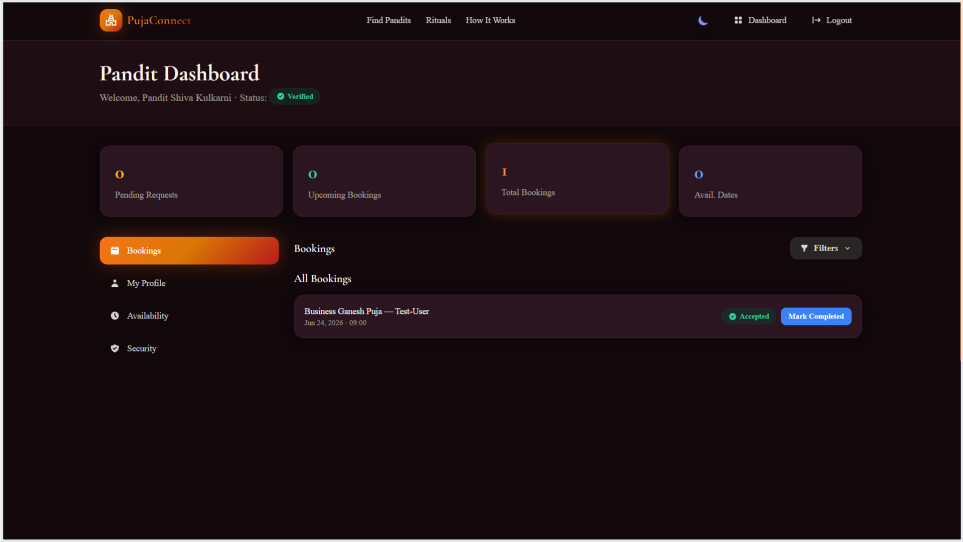
Checkout scheduler where devotees choose dates, addresses, and select active availability slots.

9. User Dashboard



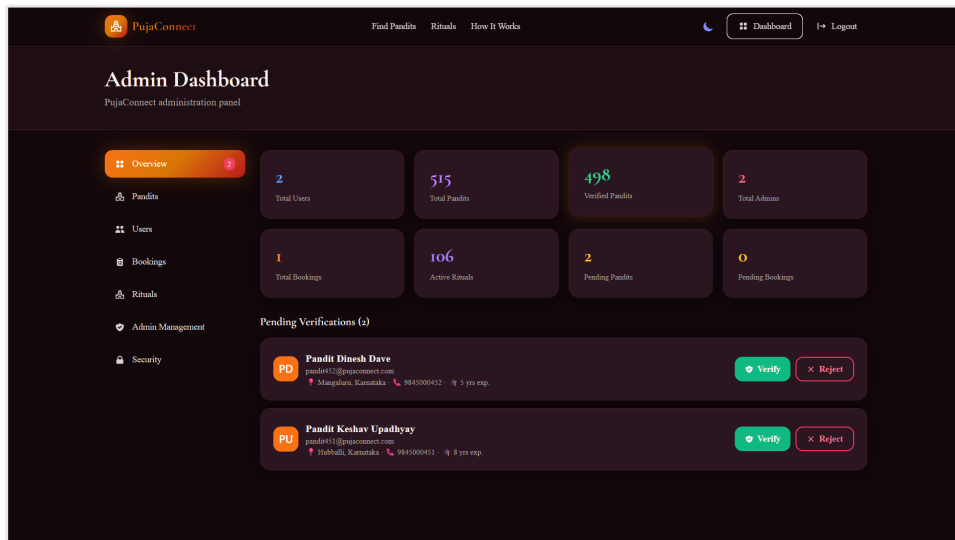
Timeline status mapping representing requested ceremonies, histories, and coordinates.

10. Pandit Dashboard



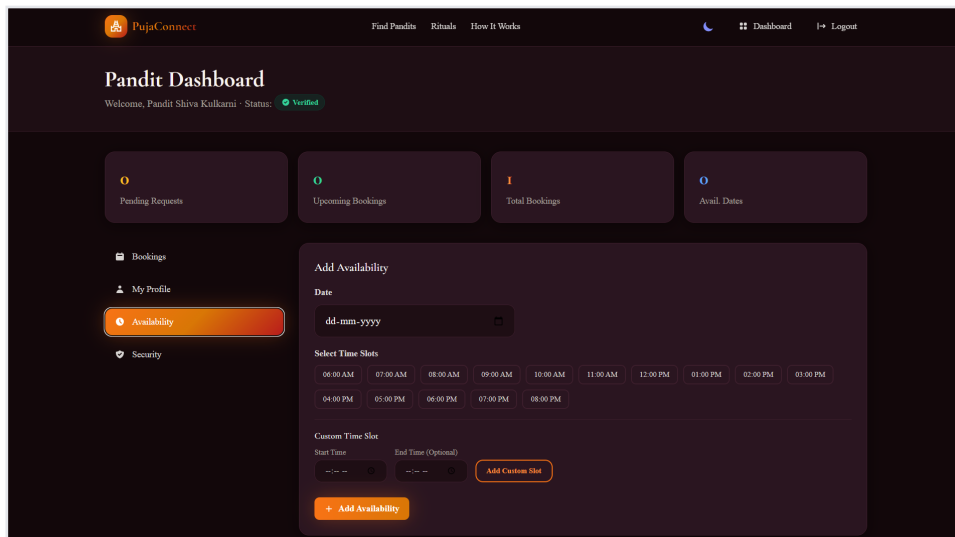
Availability scheduler calendar and client requests approval pipeline.

11. Admin Dashboard



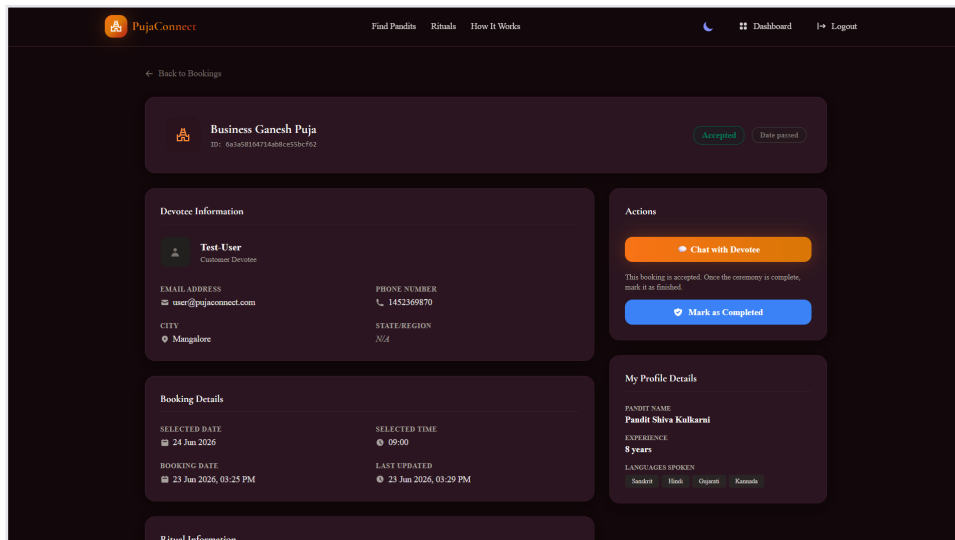
Verification review queues, catalog editor, and user suspension switches.

12. Availability Management



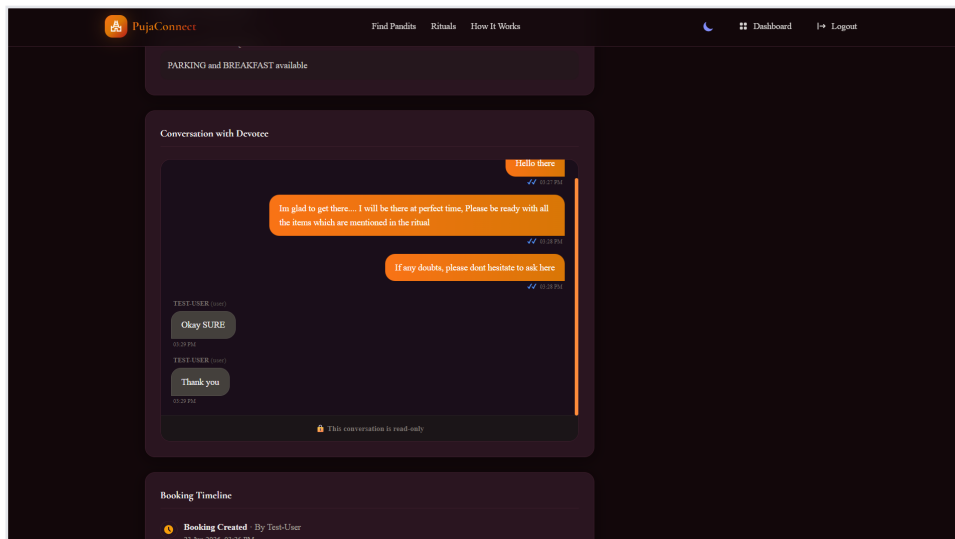
Pandits build, edit, or delete custom calendar timeframes with overlapping check guards.

13. Booking Details Page



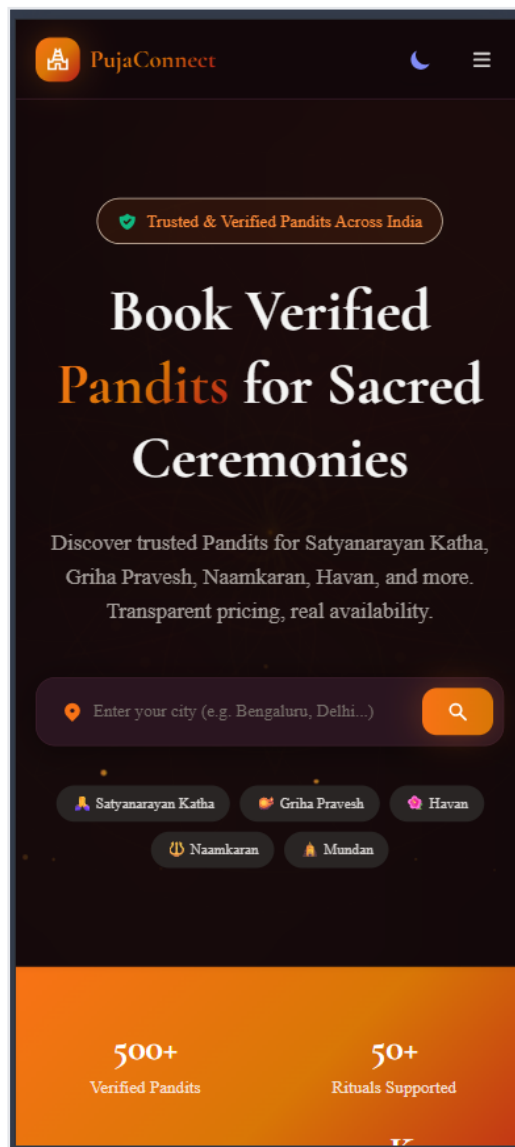
Detailed page displaying addresses, check sheets, and access coordinates.

14. Chat System



Integrated messaging interface featuring unread counters and read status indicators.

15. Mobile View



Fully responsive layout optimized for touch screens and mobile web browsers.